

CSRF — Interview Prep Notes

1. Describe CSRF logically

- Browser auto-attaches cookies/session to any request → server trusts cookie presence as proof of intent
 - Attacker forges a request from a **different origin**, victim's browser sends it with valid session cookie attached
 - App executes state change believing it's the legitimate user's action
 - Attacker never sees response (blocked by SOP) — CSRF is **blind write**, not read
 - Core flaw: server validates *who* (session) but not *whether this request was actually intended* by that user
-

2. OWASP / CWE mapping

- **CWE-352** — Cross-Site Request Forgery
 - **OWASP 2021: A01 — Broken Access Control** (moved from old A05:2017 — Broken Access Control merger)
 - Related: CWE-1275 (Sensitive Cookie w/o SameSite), CWE-346 (Origin Validation Error)
-

3. Common variants

- Classic GET-based CSRF
 - POST-based (auto-submit form)
 - JSON-based CSRF (Content-Type confusion bypass)
 - Login CSRF
 - Logout/preference CSRF (low sev, chains)
 - Multi-step/chained CSRF
 - Referrer/Origin bypass (missing, substring match, null origin via sandboxed iframe)
 - Token bypass variants (static token, unbound token, token leak via GET/Referer)
 - SameSite bypass (Lax top-level GET window, subdomain scope issues)
 - Clickjacking-assisted CSRF
 - **Stored CSRF** — payload persisted in-app (profile field, comment) rendered on victim/admin view; bypasses Origin checks since request is same-origin; overlaps CWE-79 if `<script>` used vs pure CWE-352 if HTML-tag/event-handler only
-

4. Attack surface & entry points

Sources:

- External malicious site / phishing link
- Compromised third-party site (stored XSS elsewhere)
- HTML email auto-load `<img>`
- Malicious browser extension/ad network
- Stored same-origin content
- MITM on HTTP (unencrypted)
- Related subdomain (takeover, weak SameSite scope)

Entry points:

- State-changing GET endpoints
 - Profile/account update forms
 - Financial/transactional endpoints
 - Admin/privileged action endpoints
 - Account security endpoints (password/email/2FA/session revoke) — highest priority
 - Cookie-auth API/GraphQL mutations
 - File upload/delete
 - Multi-step wizards/checkout
 - Logout endpoints (chaining)
 - OAuth callback/webhook receivers
-

5. Testing workflow (Manual / Burp / ZAP / Nuclei)

Step	Verify	Manual	Burp	ZAP	Nuclei
1. Map state-changing endpoints	Which POST/PUT/DELETE/GET routes change state	Browse app, note write actions	Site map filter by method	Spider + Sites tree	 bulk sweep from endpoint list
2. Token presence check	Is CSRF token present in form/header	View page source	Repeater — inspect request	Request Editor	N/A (manual)
3. Token removal	Is token actually validated	Strip param, resend	Repeater — delete param, send	Manual edit, send	Custom template: send w/o token param
4. Token reuse across session	Is token session-bound	Capture token Session A, use in Session B	Repeater, swap session cookie	Manual	N/A
5. Token predictability	Is token random enough	N/A	Sequencer — capture 100+ tokens, analyze entropy	N/A (no equivalent)	N/A
6. Referer/Origin bypass	Is Origin/Referer enforced correctly	Strip/spoof header	Repeater — modify Origin	Manual header edit	Template: missing/spoofed Origin matcher
7. Method tampering	GET vs POST enforcement	Change POST→GET manually	Repeater — change method	Manual	Template checking GET on state-change route
8. SameSite bypass	Cookie SameSite behavior	Cross-site nav test in browser	Manual (browser-level, not proxy)	Manual	N/A
9. PoC generation	Full exploit works end-to-end	Hand-build HTML	Built-in Burp CSRF PoC generator (Engagement tools)	No native generator — manual	N/A
10. Stored CSRF check	User-controlled field renders unsanitized on privileged view	Inject payload in profile/comment, view as admin	Repeater to inject, browser to trigger	Manual	Template for reflected payload markers

Payloads used & why:

- Empty/missing token → confirms validation is enforced, not just present

- Token from different session → confirms session-binding
 - `Origin: https://evil.com` → confirms Origin check strength
 - Auto-submit `<form>` / `fetch` → confirms real-world exploitability
-

6. Evidence to collect

- Request/response showing missing or unvalidated token
 - Working PoC HTML + screen recording/screenshot of action executing
 - Server-side confirmation of state change (before/after DB or UI state)
 - Burp CSRF PoC generator output
 - Session cookie context proof (shows victim was authenticated)
 - Sequencer entropy report (if token predictability claimed)
 - Cross-session token reuse test result
-

7. Good PoC structure

```
<html>
  <body onload="document.forms[0].submit()">

  </body>
</html>
```

- Auto-submits on load (no user click needed)
 - Demonstrates actual state change, not just request delivery
 - Attach before/after evidence of impact
 - Use Burp's built-in CSRF PoC generator for consistency in reports
-

8. Attack chains

- **CSRF token theft via CORS misconfig** → bypasses token defense entirely
- **Email change CSRF** → **password reset** → **full ATO**
- **Login CSRF** → victim's data saved into attacker's account (search history, purchases)
- **Logout CSRF** → **forced re-login as attacker session**
- **Clickjacking + CSRF** → UI redress to bypass confirmation dialogs

- **Stored CSRF** → **chains into XSS classification boundary** (script vs non-script payload)
 - **CSRF on admin panel** → **privilege escalation / full app compromise**
-

9. Common false positives

- Missing Referer due to browser privacy settings (not an attack)
 - `Referrer-Policy: no-referrer` set intentionally by app
 - Legitimate OAuth/SSO cross-origin redirect flows
 - Double-submit token reuse within same session (by design, not malicious)
 - Native mobile/API clients lacking Referer/Origin (non-browser)
 - Corporate proxy stripping headers org-wide
 - QA/Selenium automation traffic
 - Browser extensions modifying headers on legitimate user action
-

10. Prerequisites for exploitability

- No/weak/unbound CSRF token protection on target endpoint
 - Victim has valid, active session (cookie-based auth)
 - Attacker has hosting location for malicious page (or stored injection point)
 - Delivery/lure mechanism to get victim to load page
 - No effective SameSite=Strict blocking the request
 - Endpoint reachable cross-origin (state change via standard HTTP)
-

11. What attacker gains

- Unauthorized state change as the victim (data modify, delete, create)
 - Account takeover chain (email/password change)
 - Financial transaction manipulation
 - Privilege escalation if victim is admin
 - Persistence via login CSRF (victim unknowingly uses attacker's account)
 - Mass exploitation potential (single PoC works against any victim with valid session)
-

12. CIA Impact Table

Variant	Confidentiality	Integrity	Availability	Example Vector
Preference/settings CSRF	None	Low	None	C:N/I:L/A:N
Account security CSRF (email/password)	None	High	None	C:N/I:H/A:N
Financial transaction CSRF	None	High	None	C:N/I:H/A:N
Admin/privileged CSRF	None (Low if action exposes data)	High	Low–High	C:N/I:H/A:L (Scope may change)
Logout/Login CSRF	None	Low–Medium	None	C:N/I:L/A:N

- Confidentiality generally **None** — CSRF is blind (can't read response); becomes non-None only if chained with CORS/XSS
- Availability rarely impacted directly, only via chained mass-delete/lockout scenarios

13. Typical CVSS

Fixed metrics: AV:N /AC:L(or H if bypass needed) /PR:N /UI:R /S:U(or C if scope escalates)

Scenario	Vector	Score
Non-security preference change	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	4.3 (Medium)
Account email/password change	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N	6.5 (Medium/High)
Financial transaction	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N	6.5 (High-leaning via business impact)
Admin panel (new admin)	AV:N/AC:L/PR:N/UI:R/S:C/I:H/A:H	8.8 (High)

14. Business impact (financial app context)

- Direct financial loss via unauthorized fund transfer
- Account takeover → regulatory exposure (PCI-DSS, GDPR if PII involved)
- Mandatory breach disclosure if funds/PII compromised at scale
- Reputational damage — financial apps held to higher trust bar
- Chain risk: single CSRF on email-change = root of full ATO chain
- Remediation-verification cost higher for legacy/third-party-integrated endpoints

15. Detection signals (IOC)

- Missing/mismatched Referer or Origin on state-changing request
- Missing or reused/stale CSRF token
- Token mismatch vs session-bound value
- State-change immediately after session creation, no prior navigation
- Timing anomaly — request fired ms after unrelated page load (auto-submit signature)
- Spike of state-changing requests from diverse IPs targeting same victim/account
- Identical payload structure across many sessions (templated attack page)
- Content-Type mismatch (JSON endpoint receiving text/plain)

False positive filters: privacy-mode Referer stripping, corporate proxy, SSO redirects, QA automation, browser extensions — always correlate 3 signals (Origin+token+behavior), not single signal.

16. Root cause in code

```
POST /api/account/update-email
    authenticate(session_cookie)      // validates WHO
    update_email(session.user_id, request.body.new_email) // no check request was INTENTIONAL
    return success
```

- Cookie-only auth with no request-origin/intent verification
 - Contributing causes: no token generated; token generated but unchecked; token not session-bound (global/static); state change via GET; missing/None SameSite; framework CSRF middleware disabled for APIs without replacement control
-

17. Remediation & secure coding

- **Synchronizer Token Pattern:** random (CSPRNG) token, session-bound, validated server-side every state-changing request
- **Double-Submit Cookie:** token in cookie + custom header, compared server-side (works for stateless APIs/SPAs)
- **SameSite=Strict/Lax** on session cookie + `Secure` + `HttpOnly` — defense-in-depth, not sole control
- Origin/Referer validation as secondary layer (exact-match only, not substring)
- Re-authentication/step-up auth for high-sensitivity actions
- No state changes via GET (idempotent GET only)

- Prefer framework-native CSRF middleware over hand-rolled logic
 - Architectural: move to header-based Bearer/JWT auth for APIs (not auto-attached cross-origin)
-

18. Compensating controls

- WAF rule blocking missing/mismatched Origin on sensitive endpoints
 - Rate limiting on sensitive endpoints
 - Email/notification alert on sensitive account changes ("wasn't you?" link)
 - CAPTCHA on highest-risk actions
 - Shorter session timeout
 - Enforce SameSite at reverse proxy/CDN layer as stopgap
-

19. Recurring CSRF — SDLC signal

- Indicates missing/inconsistent framework-level CSRF enforcement across teams
 - Suggests devs disabling CSRF middleware for APIs without adding equivalent control
 - Points to lack of secure-by-default scaffolding/boilerplate
 - Signals gap in code review checklist and missing SAST/linter gate
 - Same systemic-framing principle as recurring IDOR — treat as control failure, not isolated bugs
-

20. Upstream prevention

- Semgrep/CodeQL rules: flag state-changing route handlers missing CSRF decorator/middleware in call chain (negative/absence-based detection, unlike IDOR's taint-based rules)
 - Rule for state-change via GET methods
 - Rule for token comparison not bound to session object
 - Framework-level default: CSRF protection enabled globally, explicit opt-out only w/ justification comment
 - Design checklist item: "Does this endpoint change state? → CSRF token + SameSite required"
 - Staged rollout: audit-only → diff-aware blocking on new code → full enforcement after legacy triage
 - Pre-commit hook for early developer feedback
-

21. Sample triage response

Confirmed CSRF on `POST /api/account/update-email`. No CSRF token present in request; endpoint accepts cross-origin POST with only session cookie for auth. Verified via PoC — auto-submitting form successfully changed victim's email without token, changed origin, or user awareness. Chains into full account takeover via password-reset flow. Rated **High (CVSS 6.5)** — Confidentiality: None, Integrity: High, Availability: None. Recommend synchronizer token pattern + SameSite=Strict as primary fix; WAF Origin-check rule as interim compensating control.

22. Sample developer remediation note

Finding: `/api/account/update-email` lacks CSRF protection — endpoint relies solely on session cookie, no token validation. **Fix:** Add framework CSRF middleware (already available via `@csrf_protect` / equivalent) to this route. Ensure token is generated per-session (CSPRNG, ≥128-bit entropy) and validated server-side against session-stored value — not just presence-checked. **Also verify:** SameSite=Lax/Strict set on session cookie; method restricted to POST only (currently confirm GET is rejected). **Retest:** Submit request without token (expect 403), submit with token from different session (expect 403), submit with valid session-bound token (expect 200).

23. Real-world examples

- **Facebook CSRF protection bypass → Account Takeover** (disclosed by researcher Sammouda) — bypassed Facebook's CSRF token validation logic, chained into full ATO
 - **Metasploit Web UI — CVE-2017-5244** — CSRF allowing termination of all running tasks in Metasploit's web project interface
 - Multiple disclosed **login CSRF** reports (e.g., forcing victims into attacker-controlled accounts, later combined with other bugs for account confusion attacks)
 - Common bug bounty pattern: password-reset-email-parameter CSRF (`?email_to=attacker@x.com`) → full admin panel compromise via reset link sent to attacker inbox
-